BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name:

ID:

> **Due Date**
>
> Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `c:\Users\Donny\OneDrive\Post-Grad\Courses\BEE 5750\Homework 4\hw4-d
   Installed MarkdownTables   v1.1.0
   Installed ArgCheck      v2.5.0
   Installed DisplayAs     v0.1.6
   Installed BenchmarkTools   v1.6.3
   Installed StatsBase     v0.34.7
   Installed PrettyTables    v3.1.0
   Installed DataFrames     v1.8.1
   Installed JuMP        v1.29.2
Precompiling project...
  15619.1 ms    ArgCheck
  10535.5 ms    StructUtils → StructUtilsTablesExt
  10579.6 ms    DisplayAs
  13306.3 ms    StatsBase
   8141.9 ms    BenchmarkTools
   3283.4 ms    MarkdownTables
  70876.7 ms    PrettyTables
 101409.5 ms    MathOptInterface
   7217.3 ms    MathOptIIS
  33606.2 ms    HiGHS
 109736.6 ms    DataFrames
   9568.1 ms    Latexify → DataFramesExt
  61561.0 ms    JuMP
 190398.0 ms    Plots
  14 dependencies successfully precompiled in 253 seconds. 208 already precompiled.
```

```julia
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

```
# Problem 2.2

function solve_and_report_farm_problem()

    # Setup data using DataFrames
    # Table 2 data
    crops = DataFrame(
        Crop = ["Soybeans", "Wheat", "Corn"],
        ProdCost = [350, 280, 390],
        SellPrice = [0.36, 0.27, 0.22]
    )
    # Table 1 data
    yields = DataFrame(
        Crop = ["Soybeans", "Soybeans", "Soybeans", "Wheat", "Wheat", "Wheat", "Corn", "Corn
        Rate = [0, 1, 2, 0, 1, 2, 0, 1, 2],
        Yield = [2900, 3800, 4400, 3500, 4100, 4200, 5900, 6700, 7900]
    )
    # Problem text data
    limits = DataFrame(
        Crop = ["Soybeans", "Wheat", "Corn"],
        PestLimit = [0.8, 0.7, 0.6],
        DemandLimit = [250000, 250000, 250000]
    )
    PesticideCost = 70
    TotalLand = 130
```

```
# Pre-process data
master_df = innerjoin(yields, crops, on = :Crop)
master_df.Profit = (master_df.Yield .* master_df.SellPrice) .- master_df.ProdCost .- (mas
master_df.VarName = master_df.Crop .* "_" .* string.(master_df.Rate)
master_df = innerjoin(master_df, limits, on = :Crop)

# Build the JuMP Model
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, Hectares[1:nrow(master_df)] >= 0)

@objective(model, Max,
    sum(master_df.Profit[i] * Hectares[i] for i in 1:nrow(master_df))
)

# Define constraints
@constraint(model, land_limit, sum(Hectares) <= TotalLand)

for crop in crops.Crop
    indices = findall(==(crop), master_df.Crop)
    @constraint(model, sum(master_df.Yield[i] * Hectares[i] for i in indices) <= master_
end

for crop in crops.Crop
    indices = findall(==(crop), master_df.Crop)
    @constraint(model, sum((master_df.Rate[i] - master_df.PestLimit[i]) * Hectares[i] fo
end

# Solve and show results
optimize!(model)

if termination_status(model) == MOI.OPTIMAL
    # Add solution to the main dataframe
    master_df.Allocation_ha = round.(value.(Hectares), digits=2)

    # 6. Plot results
    plot_data_long = master_df[:, [:Crop, :Rate, :Allocation_ha]]
    plot_data_wide = unstack(plot_data_long, :Crop, :Rate, :Allocation_ha, fill=0.0)
    x_labels = plot_data_wide.Crop
    y_matrix = Matrix(plot_data_wide[:, 2:end])
    group_labels = reshape(["Rate 0", "Rate 1", "Rate 2"], 1, 3)
```

```
        # 7. Print table
        results_table_df = master_df[:, [:VarName, :Crop, :Rate, :Allocation_ha]]
        rename!(results_table_df,
            :VarName => "Variable",
            :Rate => "Rate (kg/ha)",
            :Allocation_ha => "Allocation (ha)"
        )

        display(markdown_table(results_table_df))

        # Print optimal profit
        println("Optimal Total Profit: \$", round(objective_value(model), digits=2), "/year "

    else
        println("No optimal solution found. Status: ", termination_status(model))
    end

    return model, Hectares, land_limit, TotalLand

end

model, Hectares, land_limit, TotalLand = solve_and_report_farm_problem()
```

Variable	Crop	Rate (kg/ha)	Allocation (ha)
Soybeans_0	Soybeans	0	13.81
Soybeans_1	Soybeans	1	55.25
Soybeans_2	Soybeans	2	0.0
Wheat_0	Wheat	0	6.74
Wheat_1	Wheat	1	15.73
Wheat_2	Wheat	2	0.0
Corn_0	Corn	0	26.92
Corn_1	Corn	1	0.0
Corn_2	Corn	2	11.54

```
Optimal Total Profit: $116741.17/year
```

```
(A JuMP Model
  solver: HiGHS
  objective_sense: MAX_SENSE
    objective_function_type: AffExpr
```

```
num_variables: 9
num_constraints: 16
  AffExpr in MOI.LessThan{Float64}: 7
  VariableRef in MOI.GreaterThan{Float64}: 9
Names registered in the model
  :Hectares, :land_limit, VariableRef[Hectares[1], Hectares[2], Hectares[3], Hectares[4], H
```

```julia
# Problem 2.3

if termination_status(model) == MOI.OPTIMAL
    # Get the shadow price (dual value) of the land constraint
    land_shadow_price = shadow_price(land_limit)

    # Calculate the *estimated* profit from 10 extra ha
    profit_gain_10ha = land_shadow_price * 10

    println("Value of 1 extra ha of land (Shadow Price): \$", round(land_shadow_price, digits
    println("Estimated extra profit for 10 ha: \$", round(profit_gain_10ha, digits=2), "/yea

end
```

```
Value of 1 extra ha of land (Shadow Price): $729.4/ha/year

Estimated extra profit for 10 ha: $7294.0/year
```

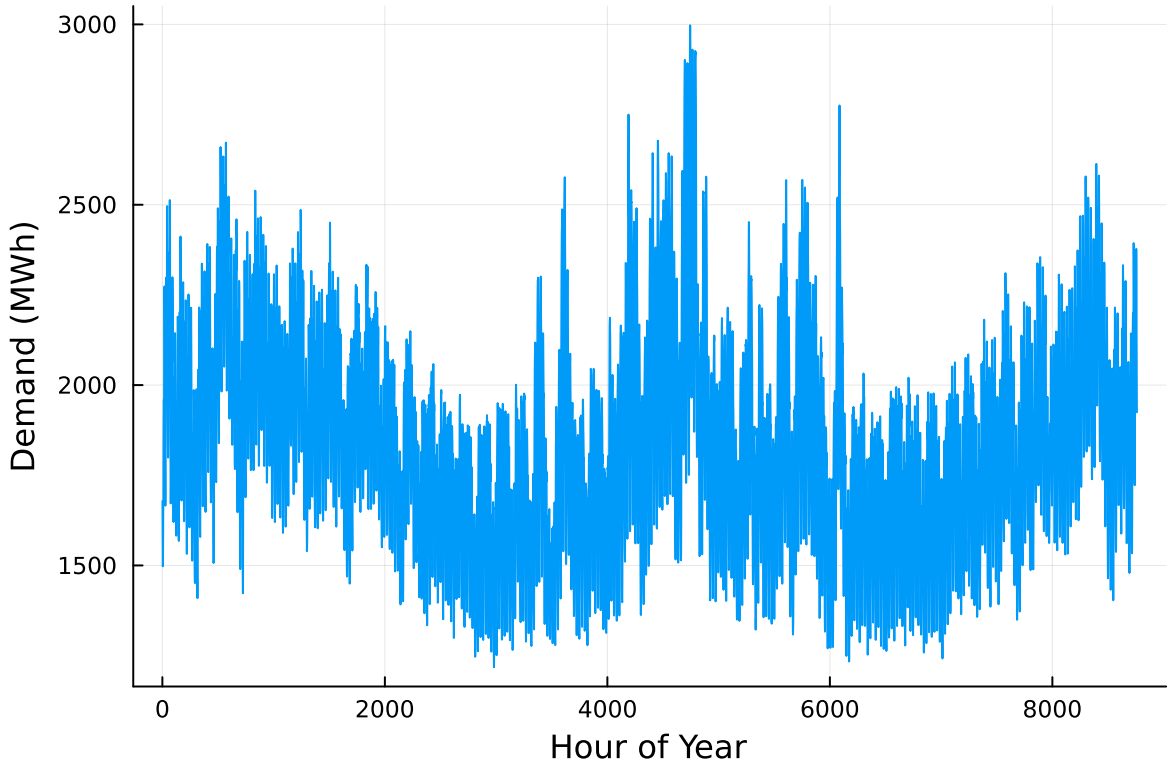
The shadow price for every additional ha of land added is \$729.4/ha/year. This value makes sense since land in this case will always be used/optimized. This means that every additional plot of land will directly translate into more profit. The value also shows to be less than the most profitable available crop + pesticide choice because it follows the constraints given (sell quota & pesticide average), which means all of the newly added plot of land will not go directly into the most profitable crops + pesticide exposure.

The most sensible price that farmers should pay for this additional plot of land is if the lease/mortgage price of the land is less than the shadow price (which already covers the operational expense). Further calculation can also be done by assuming the depreciation of the land quality, possibility of introduction of new constraints (e.g. smaller sell quota) which in this case is not accounted, yet most likely inevitable in the case of industrial farming.

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

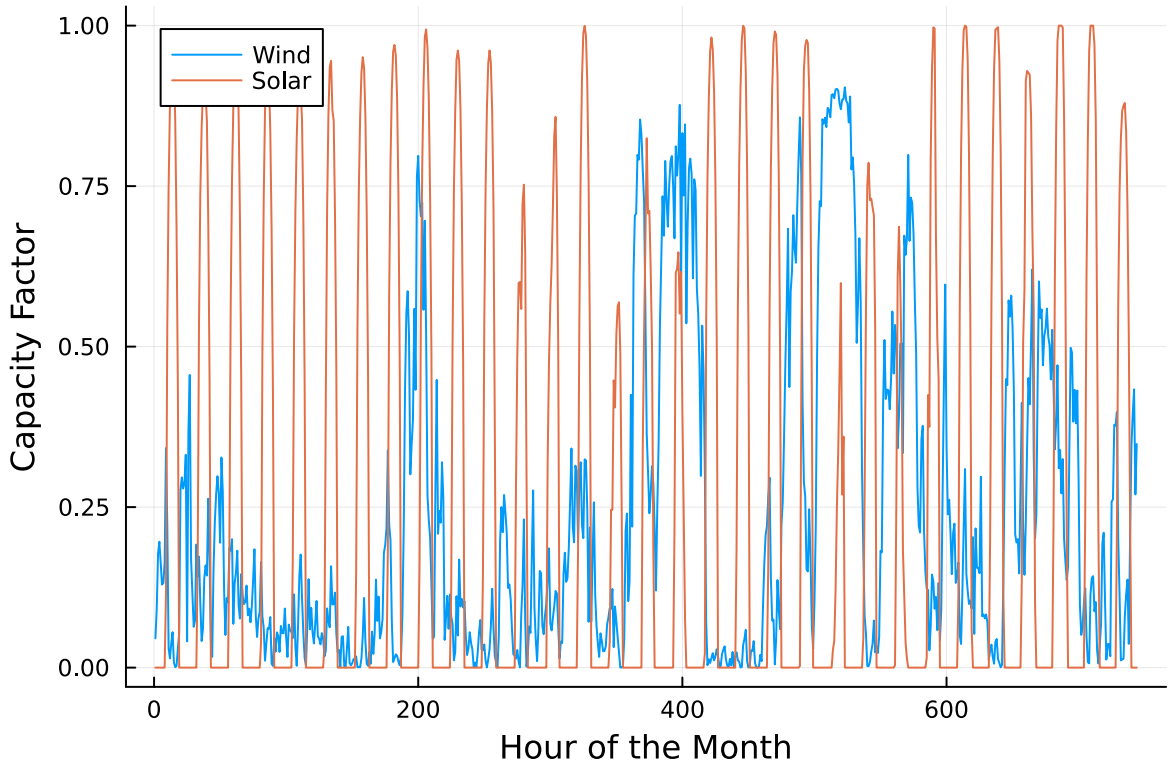
	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

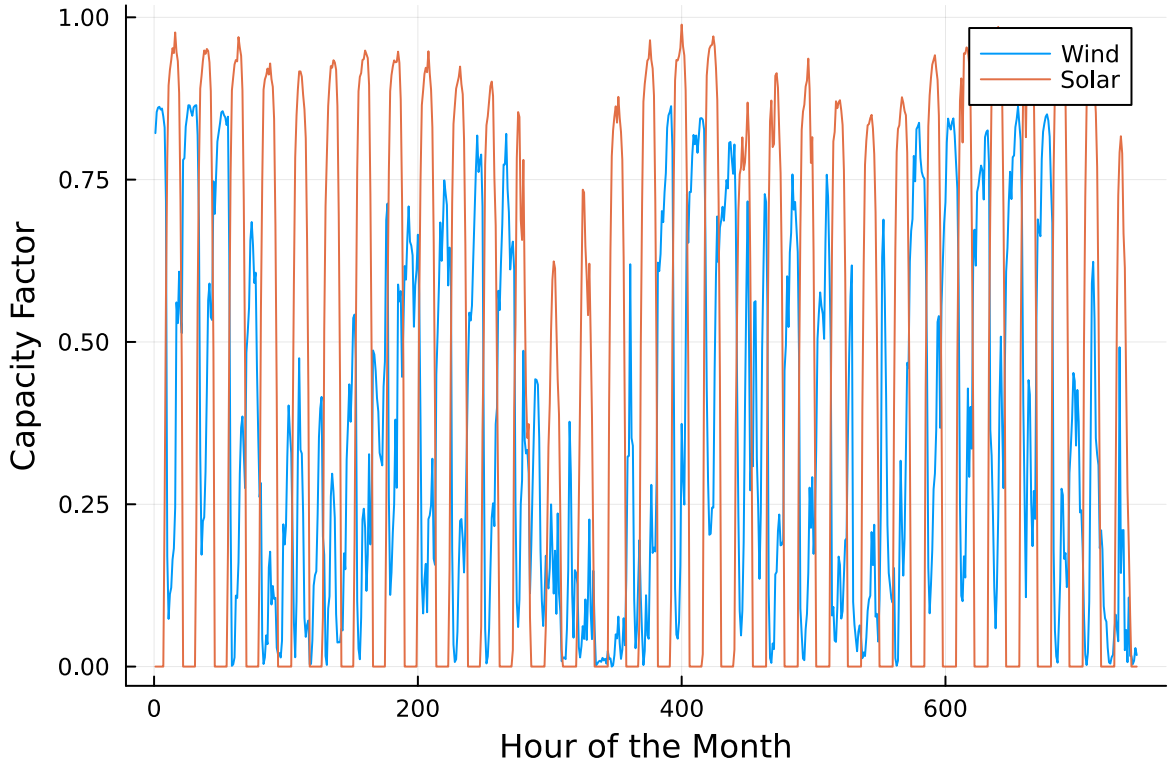
Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing

output by adding a semi-colon after the last command, or you might find that your notebook crashes.

```
# Problem 3.2

# Load necessary data to solve the problem
# Load generator data
gens_df = DataFrame(CSV.File("data/generators.csv"))
G = gens_df.Plant # The set of generators

# Load demand data (using the paths from your screenshot)
demand_df = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
D = demand_df.C # Extract the demand vector from column :C
T = 1:length(D) # The set of hours (1 to 8760)

# Load capacity factor data
cap_factor_df = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# Define parameters
# Extract data into dictionaries to mainstream lookup
FC = Dict(gens_df.Plant .=> gens_df.FixedCost)
VC = Dict(gens_df.Plant .=> gens_df.VarCost)
E = Dict(gens_df.Plant .=> gens_df.Emissions)

# Create the full Capacity Factor dictionary for all (g, t) pairs
CF = Dict()
for g in G, t in T
    if g == "Geothermal"
        CF[(g, t)] = 0.85
    elseif g == "Wind"
        CF[(g, t)] = cap_factor_df.Wind[t]
    elseif g == "Solar"
        CF[(g, t)] = cap_factor_df.Solar[t]
    else # Coal, NG CCGT, NG CT
        CF[(g, t)] = 1.0
    end
end

# Constants
P_ns = 10000.0  # Penalty for non-served demand ($/MWh)
L_CO2 = 1_500_000.0 # Annual CO2 limit (tCO2/yr)

# Build the model
```

```
model = Model(HiGHS.Optimizer)
set_silent(model)

# Decision Variables
@variable(model, Cap[G] >= 0)           # Installed capacity (MW)
@variable(model, Gen[G, T] >= 0)        # Generation (MWh)
@variable(model, Short[T] >= 0)         # Non-served demand (MWh)

# Objective Function
@objective(model, Min,
    sum(FC[g] * Cap[g] for g in G) +                    # 1. Fixed Costs
    sum(VC[g] * Gen[g, t] for g in G, t in T) +         # 2. Variable Costs
    sum(P_ns * Short[t] for t in T)                     # 3. Penalty Costs
)

# Constraints
@constraint(model, DemandBalance[t in T],
    sum(Gen[g, t] for g in G) + Short[t] == D[t]
)

@constraint(model, GenLimit[g in G, t in T],
    Gen[g, t] <= CF[(g, t)] * Cap[g]
)

@constraint(model, CO2Limit,
    sum(E[g] * Gen[g, t] for g in G, t in T) <= L_CO2
)

# Solve and show answers to questions
optimize!(model)

if termination_status(model) == MOI.OPTIMAL
    # QUESTION 1
    println("Optimal built capacity")
    for g in G
        built_mw = round(value(Cap[g]), digits=2)
        println("$(rpad(g, 12)): $(built_mw) MW")
    end
    println("\n")

    # QUESTION 2
    total_cost = objective_value(model)
```

```
    println("Total annual cost: \$$(round(total_cost, digits=0)) \n")

    # QUESTION 3
    total_non_served = sum(value(Short[t]) for t in T)
    println("")
    println("Total non-served energy: $(round(total_non_served, digits=2)) MWh")

else
    println("Error: Model did not solve to optimality. Status: ", termination_status(model))
end
```

```
Optimal built capacity
Geothermal  : 285.57 MW
Coal        : 0.0 MW
NG CCGT     : 1509.17 MW
NG CT       : 703.06 MW
Wind        : 2548.89 MW
Solar       : 2103.75 MW


Total annual cost: $7.85352959e8


Total non-served energy: 332.29 MWh
```

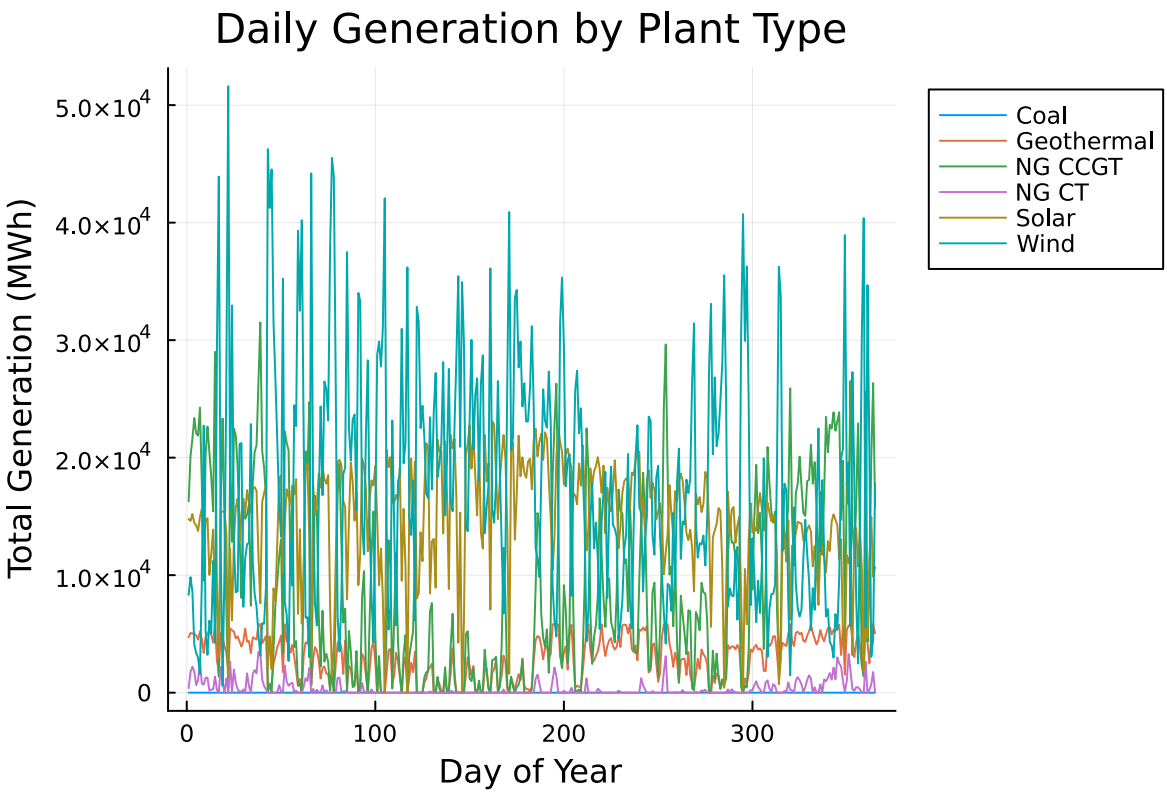
```
# Problem 3.3

println("Capacity vs. Generation Breakdown")

# Create a table to hold our results
table_header = ["Plant", "Capacity (MW)", "Cap. Fraction (%)", "Generation (GWh)", "Gen. Fra
table_data = []

# Get the total built capacity from the solved model
total_capacity = sum(value(Cap[g]) for g in G)

# Get the total annual generation from the solved model
total_generation = sum(value(Gen[g, t]) for g in G, t in T)

# Loop through each generator type
for g in G
    # Get the built capacity for this plant
```

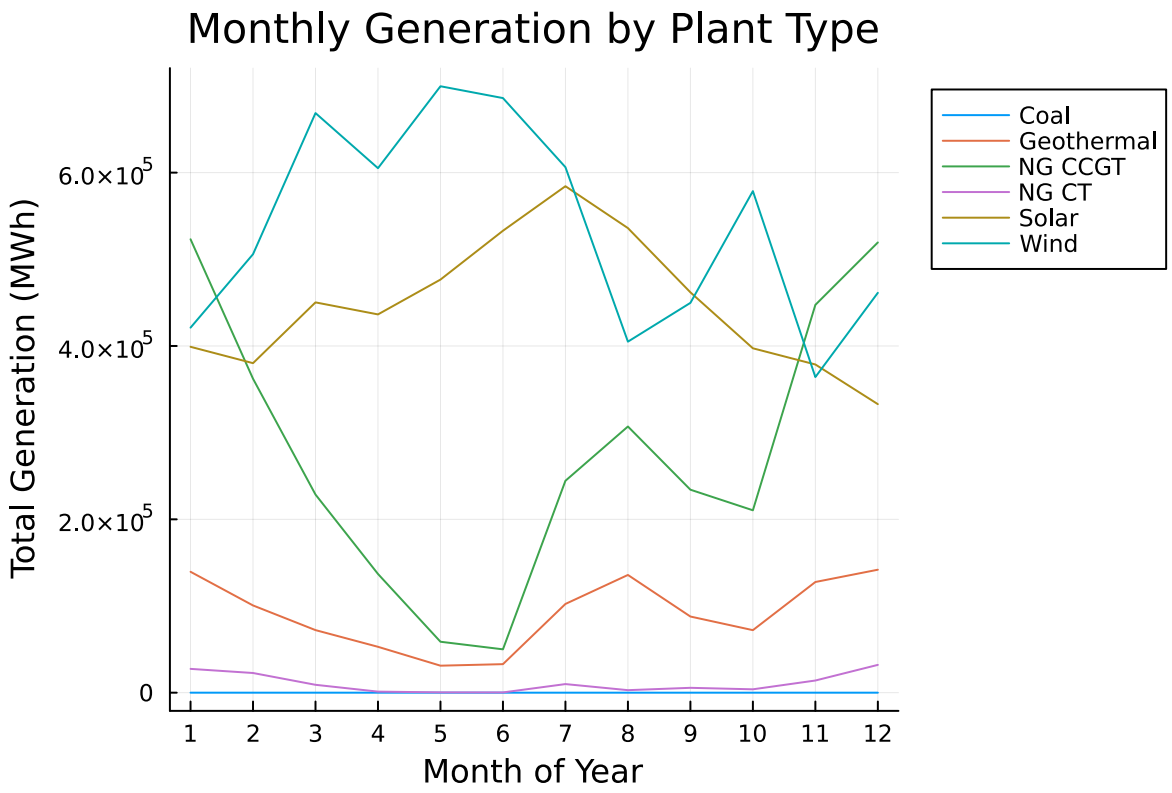
```
    built_capacity = value(Cap[g])

    # Calculate the total generation for this plant over all 8760 hours
    plant_generation = sum(value(Gen[g, t]) for t in T)

    # Calculate the fractions
    # (We include checks to prevent divide-by-zero errors)
    capacity_fraction = (total_capacity > 0) ? (built_capacity / total_capacity) : 0.0
    generation_fraction = (total_generation > 0) ? (plant_generation / total_generation) : 0

    # Add this plant's data to our table
    push!(table_data, [
        g,
        round(built_capacity, digits=1),
        round(capacity_fraction * 100, digits=1), # as percentage
        round(plant_generation / 1000, digits=1), # MWh -> GWh
        round(generation_fraction * 100, digits=1)  # as percentage
    ])
end

# Convert the collected rows into a DataFrame and display it
results_df = DataFrame(
    :Plant => [row[1] for row in table_data],
    Symbol("Capacity (MW)") => [row[2] for row in table_data],
    Symbol("Cap. Fraction (%)") => [row[3] for row in table_data],
    Symbol("Generation (GWh)") => [row[4] for row in table_data],
    Symbol("Gen. Fraction (%)") => [row[5] for row in table_data]
)

display(results_df)

# Daily generation plot
# Create a wide DataFrame to hold daily sums
daily_gen_df = DataFrame(Day = 1:365)

# Loop through each plant and add its daily generation
for g in G
    # Get the 8760-hour generation vector
    gen_vector = [value(Gen[g, t]) for t in T]

    # Reshape the vector into a (24 hours x 365 days) matrix
    hourly_matrix = reshape(gen_vector, (24, 365))
```

```
    # Sum along the 'hours' dimension (dims=1)
    # .vec() flattens the result from a 1x365 matrix to a 365-element vector
    daily_vector = vec(sum(hourly_matrix, dims=1))

    # Add this vector as a new column to the DataFrame
    daily_gen_df[!, g] = daily_vector
end

# Convert from wide format to long format
daily_long_df = stack(daily_gen_df, Not(:Day), variable_name=:Plant, value_name=:Generation)

# Create the daily plot
daily_plot = plot(
    daily_long_df.Day,
    daily_long_df.Generation,
    group = daily_long_df.Plant, # This creates one line per plant
    title = "Daily Generation by Plant Type",
    xlabel = "Day of Year",
    ylabel = "Total Generation (MWh)",
    legend = :outertopright
)
display(daily_plot)


# Monthly generation plot
# Define month lengths (2013 is not a leap year)
days_in_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

# Create a wide DataFrame for monthly data
monthly_gen_df = DataFrame(Month = 1:12)

# Loop through each plant and add its monthly generation
for g in G
    # Get the 8760-hour generation vector
    gen_vector = [value(Gen[g, t]) for t in T]

    monthly_totals = []
    current_hour = 1

    # Loop through the days in each month
    for days in days_in_month
        hours_in_this_month = days * 24
```

```
        # Get the slice of the vector for this month
        month_slice = gen_vector[current_hour : (current_hour + hours_in_this_month - 1)]

        # Sum the slice and add it to our list
        push!(monthly_totals, sum(month_slice))

        # Advance the hour counter
        current_hour += hours_in_this_month
    end

    # Add the 12-element vector of monthly totals as a new column
    monthly_gen_df[!, g] = monthly_totals
end

# Convert from wide to long format
monthly_long_df = stack(monthly_gen_df, Not(:Month), variable_name=:Plant, value_name=:Genera

# Create the monthly plot
monthly_plot = plot(
    monthly_long_df.Month,
    monthly_long_df.Generation,
    group = monthly_long_df.Plant, # One line per plant
    title = "Monthly Generation by Plant Type",
    xlabel = "Month of Year",
    ylabel = "Total Generation (MWh)",
    legend = :outertopright,
    xticks = 1:12 # Ensure the x-axis shows 1, 2, ... 12
)
display(monthly_plot)
```

	Plant	Capacity (MW)	Cap. Fraction (%)	Generation (GWh)	Gen. Fraction (%)
	String15	Float64	Float64	Float64	Float64
1	Geothermal	285.6	4.0	1097.0	6.7
2	Coal	0.0	0.0	0.0	0.0
3	NG CCGT	1509.2	21.1	3322.8	20.3
4	NG CT	703.1	9.8	129.5	0.8
5	Wind	2548.9	35.6	6451.7	39.4
6	Solar	2103.7	29.4	5366.6	32.8


Daily Generation by Plant Type
Total Generation (MWh)
5.0×10^4
4.0×10^4
3.0×10^4
2.0×10^4
1.0×10^4
0
0
100
200
300
Day of Year
Coal
Geothermal
NG CCGT
NG CT
Solar
Wind



`Capacity vs. Generation Breakdown`

While most of the plants have a minor difference between its capacity fraction and generation fraction (+/- 2-4%), NG CT shows a significant difference with 9% difference between capacity and generation fraction. The contribution of the plant is closer to 0% in its implementation.

To understand this phenomena, we can see it from the objective function and constraints that can be inferred from the monthly generation chart above. This model prioritize on the least total cost possible for the combination of plants, while adhering to several restrictions such as CO2 emission and physical limits. Based on this, wind and solar will become the most favorite option due to its relatively moderate capital cost and 0 operational cost. This means the model will prioritize these plants with consideration to its fluctuation. The next in-line is the NG CCGT plant, this becomes the third option, or basically the main backup to the favorites. NG CCGT is picked because its ability to be consistent, while not having exorbitant total cost compared to geothermal and less environmental footprint compared to NG CT. This is why during the peak generation ability of wind and solar, the dispatch from NG CCGT becomes low. The next option is geothermal, which has a high capital, but 0 cost to operate. Geothermal becomes the fourth option because of it does not add more cost to generate and becomes a buffer to the CO2 emitted by the NG CCGT plant. The last is the

NG CT plant, which as mentioned above has the most significant difference from the capacity. Based on the graph, it can be inferred that NG CT becomes the just in case option. Meaning it will only be leveraged only and only when other plants are already leveraged.

The minimum generation by NG CT is also influenced by the penalty constraint. The model will also calculate whether paying the penalty for non-served demand makes more economic sense compared to generating electricity from the NG CT plant. Given that the model has a non-served demand, it means that there are some points where paying fines makes more economic sense compared to opearting the NC GT.

To sum it up, the difference between capacity and generation fraction for NC GT makes sense for this context. In the model where lowest cost is the main goal, operating a plant with the highest operating cost will be less desirable, even when fines is the consequence of it.

```
# Problem 3.5

# Shadow price of CO2
co2_shadow_price = shadow_price(CO2Limit)
println("The shadow price of CO2 is: \$$(round(co2_shadow_price, digits=2)) per tCO2 \n")

# Value of allowing 1000 more tCO2
additional_emissions = 1000
value_of_extra_emissions = co2_shadow_price * additional_emissions
println("The value (cost savings) of allowing an additional 1000 tCO2/yr is: \$$(round(value
```

```
The shadow price of CO2 is: $-182.77 per tCO2

The value (cost savings) of allowing an additional 1000 tCO2/yr is: $-182771.94
```

Based on the JuMP model above, $187.77 can be saved for every relaxed tCO2 limit. This means that annually, $187772 can be saved for every additional 1000 tCO2 allowed per year.

However, it is important to note that this saving is strictly under the assumption of the data given. Adding new facts, assumption, and constraints can substantially alter the result given.

References

Other student consulted: Jinrong Yang (Problem 2 and 3)

LLM was incorporated in solving problems given.

1. General: LLM was used to understand terms from several libraries such as MOI.OPTIMAL and ":= [name]" format

2. Problem 2.2-2.3: LLM was used after deciding the decision variable, objective function, and constraints to build the loop (for and if) function in the model
3. Problem 3.2-3.5: similar to Problem 2, LLM was utilized to build the loop functions in the model after deciding the decision variable, objective function, and constraints in the model